

Hands-On GNN Chapter 1-2

Getting Started + Graph Theory

A7610 그래프신경망특론

March 11, 2026

- 1 Chapter 1: Getting Started with Graph Learning
- 2 Chapter 2: Graph Theory for Graph Neural Networks
- 3 마무리

왜 그래프인가?

- 그래프는 노드(객체)와 엣지(관계)를 함께 모델링
 - 노드: 엣지 없는 노드는 전통적인 데이터 표현으로도 가능
 - 엣지: 반드시 2개 이상의 노드 포인트를 포함하며 관계 정보를 표현
- 관계 정보가 핵심인 도메인에서 강력
- 예: 소셜/교통/생물/추천 시스템
- GNN: 관계 구조를 활용한 딥러닝 모델링 프레임워크

Left: 노드 중심 표현, Right: 관계 중심 표현 (그래프)

Tabular dataset

ID	Name	Age	Gender
1	Mary	76	Female
2	John	75	Male
3	Kate	46	Female
4	Robert	47	Male
5	Loise	18	Female

Graph dataset

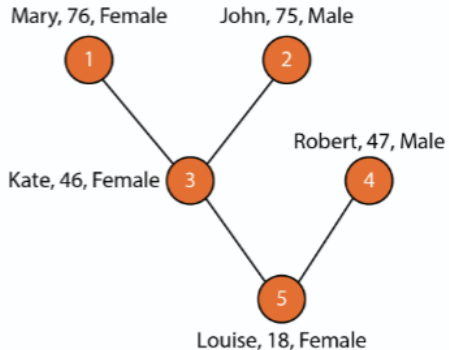


Figure 1.3 – Family tree as a tabular dataset versus a graph dataset

관계 정보(엣지)의 테이블 표현

Node Table

node_id	name	age	gender
1	Mary	76	Female
2	John	75	Male
3	Kate	46	Female
4	Robert	47	Male
5	Loise	18	Female

Edge Table

edge_id	src_node_id	dst_node_id
1	1	3
2	2	3
3	3	5
4	4	5

- 엣지 한 행은 반드시 두 노드를 가리키는 `src_node_id`, `dst_node_id` 두 포인터를 가진다.
- 또한 연결 자체에 대한 별도의 정보를 가질 수 있다.
(예: weight, relation category 등)

Graph Learning의 대표 과제

- **Node Classification:** 각 노드의 라벨(클래스)을 예측
- **Link Prediction:** 두 노드 사이의 연결 존재 여부를 예측
- **Graph Classification:** 그래프 전체(분자/문서/세션)에 대한 라벨을 예측
- **Graph Generation:** 조건을 만족하는 새로운 그래프 구조를 생성

예: 사용자 관심사 분류(Node), 친구 추천(Link), 분자 독성 분류(Graph), 신규 분자 설계(Generation)

GNN의 구조

- 입력: 그래프 구조 데이터
- GNN은 노드 특징과 관계 구조를 함께 활용해 표현 학습
- 출력: 노드/엣지/그래프 수준의 예측 등 문제 유형에 따라 다양한 태스크 수행

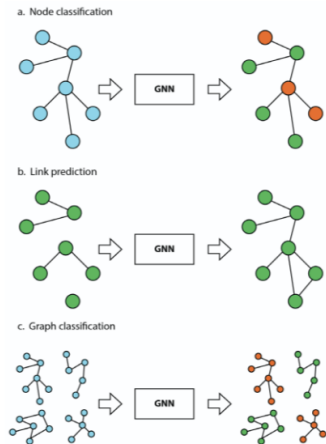
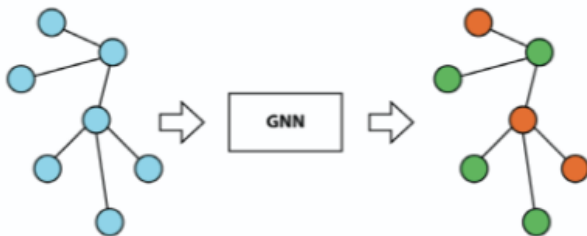


Figure 1.4 – High-level architecture of a GNN pipeline, with a graph as input and an output that corresponds to a given task

문제 1: Node Classification

a. Node classification

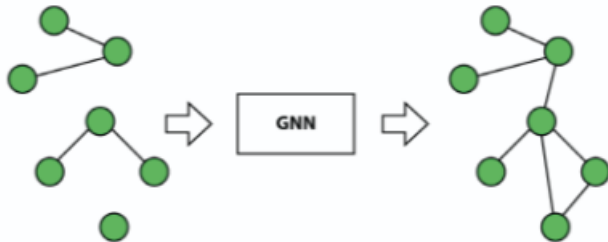


목표: 각 노드의 라벨을 예측
예시

- SNS 사용자 분류 - 예: 관심사/성향/스팸 여부
- 논문 주제 분류 - 예: AI/시스템/이론 등
- 금융 네트워크에서 사기 계좌 탐지 - 예: 정상/사기 계좌

문제 2: Link Prediction

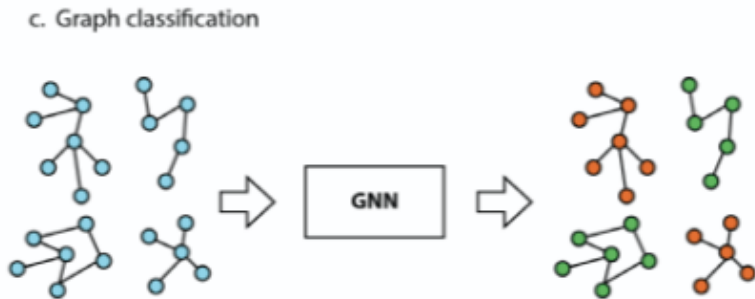
b. Link prediction



목표: 아직 링크가 없는 두 노드 사이의 링크가 생길 가능성 예측
활용 예시

- SNS 친구 추천 - 예: A와 B가 친구가 될 가능성
- 연구자 협업 예측 - 예: 연구자 A와 B가 공동 연구할 가능성
- 추천 시스템 - 예: 사용자 A가 상품 B를 구매할 가능성

문제 3: Graph Classification



목표: 그래프 전체에 대한 라벨 예측 - 그래프 전체를 하나의 객체로 보고 특정 태스크 수행 활용 예시

- 분자 구조의 독성 예측 - 예: 분자 A가 독성이 있는지 여부
- 화합물의 약효 예측 - 예: 화합물 A가 특정 질병에 효과가 있는지 여부
- 프로그램 코드 분석 - 예: 코드 A가 버그가 있는지 여부

Graph Learning이 빠르게 발전한 이유

- 그래프 학습은 다양한 도메인에 적용 가능해 연구/개발 가치가 큼
- 최근 발전의 핵심 동력:
 - 대규모 데이터셋 가용성 증가
 - 고성능 컴퓨팅 자원 확장
 - ML/AI 방법론의 빠른 발전

대표 기술 계열 4가지

- 1 Graph Signal Processing
- 2 Matrix Factorization
- 3 Random Walk
- 4 Deep Learning

4가지 Graph Learning Technique Family

- **Graph Signal Processing:** 그래프 푸리에 변환/스펙트럴 분석으로 연결 구조의 내재 특성을 분석
- **Matrix Factorization:** 큰 관계 행렬을 저차원 잠재 표현으로 분해해 패턴을 compact하게 표현
- **Random Walk:** 그래프 위 탐색 경로를 시뮬레이션해 노드 간 관계 정보를 수집하고 학습 데이터 생성
- **Deep Learning:** 다층 신경망으로 그래프를 벡터로 인코딩해 다양한 다운스트림 과제에서 높은 성능 달성

실무에서는 이 기법들이 서로 배타적이라기보다, 혼합되어 함께 사용되는 경우가 많다.

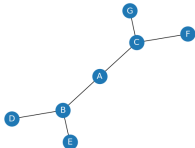
왜 GNN인가?

- 기존 ML과의 차이: 기존 ML은 노드 feature에만 집중하는 반면, GNN은 노드 특징과 관계 구조를 함께 활용
- 기존 그래프이론과의 차이: 그래프 이론은 주로 그래프의 수학적 성질과 알고리즘에 초점, GNN은 그래프 구조를 활용한 표현 학습과 예측에 초점
- 이웃 정보를 반복 집계(message passing)
- 노드 특징 + 관계 구조를 함께 반영
- 대규모/고복잡도 문제에서 표현학습에 유리
- 기존 graph ML은 handcrafted features + shallow models이 많았지만, GNN은 end-to-end 학습 가능한 deep architecture로 발전
- GNN은 complex graph data에서 높은 표현력과 일반화 능력을 보여주며, 다양한 도메인에서 state-of-the-art 성능 달성
- 하지만 GNN은 충분한 데이터가 필요, 복잡한 문제에는 효과적이나 작은 데이터셋의 간단한 문제는 기존 ML이 더 적합할 수 있음

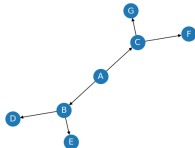
그래프의 기본 성질

- Directed vs Undirected
- Weighted vs Unweighted
- Connected vs Disconnected

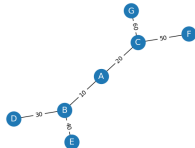
Undirected



Directed



Weighted



Disconnected example

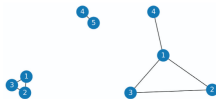
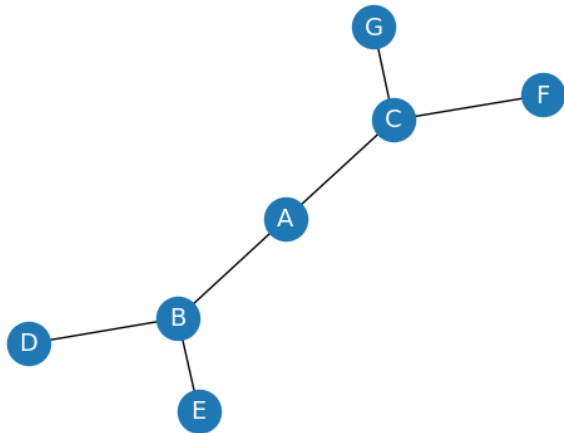


Figure 2.4 – Left: graph 1 with isolated nodes (disconnected graph); right: graph 2 where each node is connected to at least another one (connected graph)

같은 노드 집합이라도 edge의 방향, 가중치, 연결성에 따라 그래프의 의미와 해석이 달라진다.

네트워크 구조



- Chapter 2 노트북에서 `networkx`로 만든 무방향 그래프 결과
- 중심 노드 A에서 B, C로 연결되고, 다시 하위 노드로 퍼지는 트리형 구조
- 이 한 장의 그림으로 node, edge, degree, path 개념을 함께 설명할 수 있다
- 이후 GNN에서는 이런 연결 구조를 따라 이웃 정보가 전달된다

NetworkX 코드 예시: 그래프 생성과 해석

```
import networkx as nx
import matplotlib.pyplot as plt

# 무방향 그래프를 하나 만든다.
G = nx.Graph()

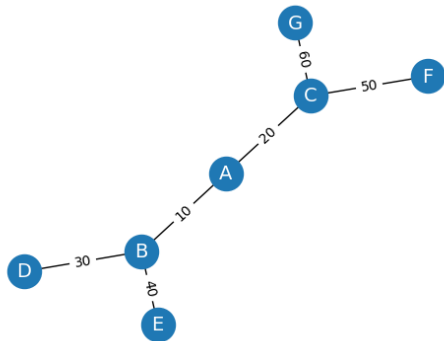
# 각 튜플은 두 노드 사이의 연결(edge)을 뜻한다.
G.add_edges_from([
    ("A", "B"), ("A", "C"),
    ("B", "D"), ("B", "E"),
    ("C", "F"), ("C", "G"),
])

# 노드 A의 차수(degree)는 연결된 이웃의 수다.
print(f"deg(A) = {G.degree['A']}")

# 시각화용 배치를 계산한 뒤 그래프를 그린다.
pos = nx.spring_layout(G, seed=0)
plt.axis("off")
nx.draw_networkx(G, pos=pos, node_size=600, font_color="white")
```


코드가 그래프 이론과 연결되는 방식

- `nx.Graph()`는 방향이 없는 그래프를 만든다
- `add_edges_from()`은 edge list 표현을 코드로 옮긴 것이다
- `G.degree['A']`는 노드 A의 degree를 바로 계산한다
- `spring_layout`은 연결된 노드가 가깝게 보이도록 배치한다



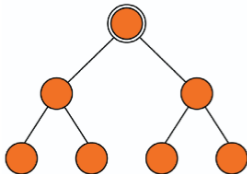
가중치 그래프를 쓰면 edge마다 중요도나 비용을 추가로 해석할 수 있다.

중요 그래프 유형

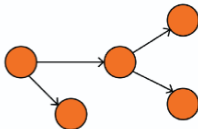
- Tree / Rooted Tree: 트리는 사이클이 없는 연결 그래프. 루트 트리는 트리의 한 노드를 루트로 지정한 그래프
- DAG: 유향+사이클 없는 그래프. 트리보다 일반적이며, 순서/의존성 표현에 유용
- Bipartite Graph: 노드 집합이 두 개로 나뉘고, 같은 집합 내에서는 엣지가 없는 그래프. 예: 사용자-상품 네트워크
- Complete Graph: 모든 노드가 서로 연결된 그래프. 예: 소셜 네트워크에서 모든 사람이 친구인 경우

각 유형은 특정 도메인과 문제 유형에서 자주 등장하며, GNN 모델링 시에도 구조적 제약으로 활용될 수 있음

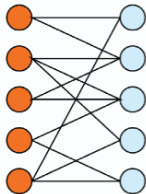
Rooted tree/tree



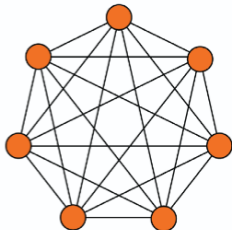
Directed acyclic graph



Bipartite graph



Complete graph



- Degree: $\deg(v)$, Directed(-:in, +:out): $\deg^-(v), \deg^+(v)$
- Path, Cycle, Acyclic
- Centrality:
 - Degree centrality
 - Closeness centrality
 - Betweenness centrality
- Density:

$$\rho_{\text{undir}} = \frac{|E|}{|V|(|V| - 1)/2}, \quad \rho_{\text{dir}} = \frac{|E|}{|V|(|V| - 1)}$$

IPYNB 예제: Degree 계산

```
G = nx.Graph()
G.add_edges_from([
    ("A", "B"), ("A", "C"),
    ("B", "D"), ("B", "E"),
    ("C", "F"), ("C", "G"),
])

# 무방향 그래프에서 는degree
# 해당 노드에 연결된 이웃 수다.
print(f"deg(A) = {G.degree['A']}")

DG = nx.DiGraph()
DG.add_edges_from([
    ("A", "B"), ("A", "C"),
    ("B", "D"), ("B", "E"),
    ("C", "F"), ("C", "G"),
])

# 방향 그래프에서는 in/out 를degree 구분한다.
print(f"deg^-(A) = {DG.in_degree['A']}")
print(f"deg^+(A) = {DG.out_degree['A']}")
```

실행 결과

```
deg(A) = 2
deg^-(A) = 0
deg^+(A) = 2
```

- A는 B, C와 연결되어 degree가 2
- 방향 그래프에서는 들어오는 엣지와 나가는 엣지를 분리해 본다

IPYNB 예제: Centrality 비교

```
# 같은 그래프에서 대표 를 centrality 계산한다.
print(f"Degree centrality      = "
      f"{nx.degree_centrality(G)}")
print(f"Closeness centrality    = "
      f"{nx.closeness_centrality(G)}")
print(f"Betweenness centrality = "
      f"{nx.betweenness_centrality(G)}")
```

노트북 출력에서도 A, B, C의 centrality 값이 D, E, F, G보다 높게 나온다.

핵심 해석

- Degree centrality: 직접 연결이 많은 노드가 큼
- Closeness centrality: 다른 노드까지 평균 거리가 짧을수록 큼
- Betweenness centrality: 최단경로를 자주 중개하는 노드가 큼
- 이 예제에서는 중심부에 있는 A, B, C가 말단 노드보다 더 중요하게 측정된다

IPYNB 확장 예제: Density 계산

```
# 같은 무방향 그래프의 밀도를 계산한다.  
num_nodes = G.number_of_nodes()  
num_edges = G.number_of_edges()  
  
print(f"|V| = {num_nodes}, |E| = {num_edges}")  
  
# networkx 를 density 바로 계산해 준다.  
print(f"density = {nx.density(G):.3f}")
```

실행 결과

$|V| = 7, |E| = 6$
density = 0.286

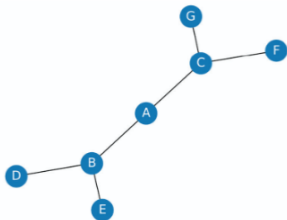
- 가능한 전체 연결 수 대비 실제 연결 수의 비율
- 희소 그래프일수록 density가 낮다
- 실제 그래프 데이터는 대개 dense 보다 sparse에 가깝다

그래프 표현 방식 비교

표현	장점	단점
Adjacency Matrix	엣지 조회 $O(1)$	공간 $O(V ^2)$
Edge List	저장 단순, 희소 그래프 유리	엣지 조회 느림
Adjacency List	공간 $O(V + E)$	엣지 조회는 Matrix보다 느릴 수 있음

Adjacency Matrix 설명

- 인접 행렬은 노드 간 연결 여부를 $|V| \times |V|$ 행렬로 저장한다
- 노드 i 와 j 가 연결되어 있으면 $A_{ij} = 1$, 아니면 0
- 가중치 그래프라면 0/1 대신 weight 값을 넣을 수 있다
- 장점: 특정 엣지 존재 여부를 즉시 확인 가능
- 단점: 희소 그래프에서도 공간이 많이 든다



A	...					G
0	1	1	0	0	0	0
1	0	0	1	1	0	0
1	0	0	0	0	1	1
0	1	0	0	0	0	0
0	1	0	0	0	0	0
0	0	1	0	0	0	0
0	0	1	0	0	0	0

IPYNB 예제: Adjacency Matrix / Edge List / Adjacency List

같은 그래프를 세 가지 표현으로 저장할 수 있다.

```
adj = [[0,1,1,0,0,0,0],
        [1,0,0,1,1,0,0],
        [1,0,0,0,0,1,1],
        [0,1,0,0,0,0,0],
        [0,1,0,0,0,0,0],
        [0,0,1,0,0,0,0],
        [0,0,1,0,0,0,0]]

edge_list = [(0, 1), (0, 2), (1, 3),
              (1, 4), (2, 5), (2, 6)]

adj_list = {
    0: [1, 2],
    1: [0, 3, 4],
    2: [0, 5, 6],
    3: [1], 4: [1], 5: [2], 6: [2]
}
```

- adj: 엣지 존재 여부를 표처럼 확인 가능
- edge_list: 저장이 단순하고 직관적
- adj_list: 실제 이웃 탐색에 가장 자주 쓰임

같은 그래프라도 목적에 따라 표현 방식을 다르게 선택한다.

BFS vs DFS

BFS

- Queue
- 레벨 순회
- 무가중치 최단경로 유리

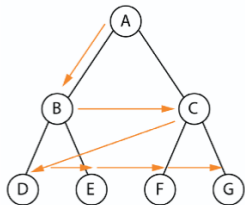


Figure 2.7 – Example of graph traversal made by a breadth-first search

DFS

- Stack/Recursion
- 깊이 우선 + 백트래킹
- 사이클 탐지/위상정렬 기반

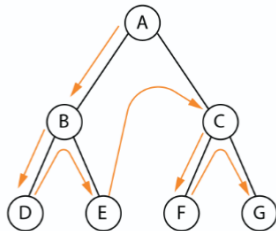


Figure 2.8 – Example of graph traversal made by a depth-first search

시간복잡도: BFS/DFS 모두 $O(|V| + |E|)$

BFS와 DFS 알고리즘

```
def bfs(graph, node):
    visited, queue = [node], [node]
    while queue:
        node = queue.pop(0)
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.append(neighbor)
                queue.append(neighbor)
    return visited

def dfs(visited, graph, node):
    if node not in visited:
        visited.append(node)
        for neighbor in graph[node]:
            visited = dfs(visited, graph, neighbor)
    return visited
```

노트북 실행 결과

```
bfs(G, 'A')
['A', 'B', 'C',
 'D', 'E', 'F', 'G']
```

```
dfs([], G, 'A')
['A', 'B', 'D',
 'E', 'C', 'F', 'G']
```

- BFS는 가까운 레벨부터 넓게 방문
- DFS는 한 경로를 끝까지 따라간 뒤 되돌아온다

1, 2장 핵심 요약

- ① 그래프 학습의 본질은 관계 구조 활용
- ② GNN의 본질은 이웃 집계 기반 표현학습
- ③ 그래프 이론(표현/탐색/측도)은 GNN 이해의 필수 기초

감사합니다!